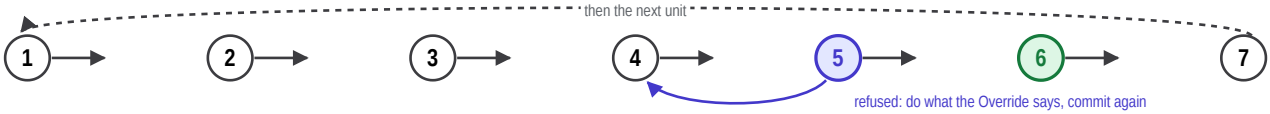
Charter at the desk

Package 0.4.11. Front: the working day and three flows. Back: the reference.
`charter` = `python3 .charter/bin/charter` (project root), `charter-pkg` = `python3 ~/Projects/charter/bin/charter-pkg`

O the owner acts · A the agent acts · *refused* hard gate, names its override · *warns* logged, lets it pass · *passes* · *" phrase "* what you say to the agent

Every day: the unit loop

One unit per deliverable. The agent runs steps 1 to 7; the teal lane is yours. Source: `docs/daily-use.md` § The loop.



Open a session

Read the banner:

```
charter 0.4.11, track prototype, opt-ins: none
```

- `charter next`: active unit, road, next units, what waits on you. Writes nothing.
- Hook not active: `charter setup` once.
- On 0.4.2 or older you see no banner: `charter config`.

Pick or write the unit

"start unit <id>" or *"write a unit for …"* (skill `charter-unit`).

Ready when each **AC** names a probe that **fails now** (`(guard)` if it protects what works), `perimeter:` lists the paths, `scope:` is set on client, `rail:` where there are rails.

Start it

```
charter unit start <id>
charter probe <id>
```

```
→ exit 1 (ModuleNotFoundError …)
```

That failure is the **baseline**, the session's first `## Log` line. A probe passing now proves nothing.

Work

- Outside `perimeter:`? Add `- path: why` under `## Outside perimeter`.
- *"record the decision"*: `charter/decisions/<slug>.md`, cited in code as `charter:<slug>`.
- Frozen file (800 lines; 400 for governance): move code out.
- Unsold work: a change request.
- Dated `## Log` line when something lands.

Commit

On a branch. The hook runs `charter check --staged`:

```
charter check: 0 blocking, 0 warning(s) [staged, …]
```

Refused: each line ends with **Override:**; do that. **Never** `--no-verify` or a Bash workaround: CI runs the same checks.

Close it

"finish the unit"

1. `charter probe <id>` → exit 0
2. tick `- [x] AC-n`
3. `## Evidence`: SHA, `path:line`, the `$ command` lines
4. `status: done`, closing log line
5. `charter unit stop`

Final message cites the same, or the Stop hook blocks it once.

Report

"where does this stand?" (skill `charter-progress`)

`charter ledger fold`
`charter synth --write`
Writes `charter/progress.md`, or refuses if a line is unanchored. Never hand-edited. A client gets scope and accepted units.

O Only you

Commit what is handed over (install, config, settings): paste the command after `!` in Claude Code, or run it in your own terminal; `charter handover` prints it again.

Re-plan: add or reorder rows in `charter/rails.md`; `charter rail <id> done` closes one.

Rule a proposal: `charter rule <id> active|rejected`, or `rule <old> superseded --by <new>`. **Approve** a change: `charter approve <id> [--reject]`. Each commits itself.

Push, open, merge with a merge commit, which keeps evidence SHAs. **Deploy.**

Let a frozen file grow: a `kind: split` unit, or a `size_exceptions` entry (budget, `because:`, `expires:`).

Your queue: `charter owner`, each item with its command, a chain's wait included. Answer the agent's commitments (price, dates, data, releases).

Does the project already have code, taskets or documents?

no: install directly · yes: survey first

Install steps 1 and 2, either way: client track only under a signed devis or contract, else prototype; --track is required.

A new project

Say *"install charter on <project>"*. The agent follows `docs/install.md` and stops with the files staged. `$P` is the project; counts are from a 0.4.11 rehearsal.

0. **Preconditions.** A git repo with a commit; a clean tree; no other session in it; `charter-pkg status $P` → `installed: none`.
3. **Branch.** `git -C $P switch -c charter/install`, never the main line.
4. **Dry run.** `charter-pkg install $P --track prototype --dry-run` → `dry run: 45 change(s)`. Watch for `KEEP git hooks` (step 6) and `IGNORED by` (ask the owner).
5. **Install.** The same without `--dry-run`. Run again: `already installed … nothing to do`.
6. **Only after KEEP git hooks:** add `python3 -B .charter/bin/charter check --staged` to the lefthook, pre-commit or husky hook.
7. **Staged by the installer**, by name (44 files); a step 6 file you stage by name. Never `git add -A`.
8. **You commit.** Set `"owner"` in `charter/config.json` to your git author name; `charter config` shows no `problem:`; run the handed-over `git -C $P commit … --<paths>`. The agent's attempt gets 2 blocking (settings, config): expected.
9. **Verify.** `charter check --all` → 0 blocking; `charter-pkg status $P` → 0 modified, missing, differing; a new session shows the banner; a fresh clone passes `check --all --strict`.

O You push and merge (merge commit). Each other clone: `charter setup` once. Client track: you write `charter/scope.md` from the signed devis, one item per sold deliverable, no prices.

Later, a new version: `charter-pkg fleet --upgrade` upgrades each project behind, commits it when only package files changed, hands over the rest. One by hand: `docs/install.md` § Upgrade. Rollback: `git revert`.

Sources: `docs/install.md` steps 0-9, § Every other clone, § Upgrade; `docs/migration.md` Paths A and B; the 0.4.11 counts: an install into an empty repository, 2026-10-06.

Migrating an existing project

Say *"put charter on <project>"*: `CharterAdmin` runs its `Migrate` workflow. Nothing touches the project before your go-ahead.

1. **Survey, read-only.** `charter-pkg survey $P --track prototype --out <dir outside>`: `survey.md`, `plan.md`, an install rehearsed in a throwaway clone. `--test-cmd` exit 0: probes can run at commit.
2. **Settle what the facts settle.** A lookup is not a decision; a proposal is not a scope.
3. **Your calls on a Slate**, including the go-ahead for the real tree.
4. **Rehearse the whole plan** in a kept clone (`--keep-clone`), the only place `CHARTER_OWNER=1` is allowed.
5. **Execute** in a worktree on `charter/install`: one step, one commit, one check; red stops it; owner-only steps wait staged.
6. **You push and merge.** The agent reports each step with its check, and what is left.

The old work: a tasket listed in `charter/legacy-taskets.txt` closes with a warning only; units done before show as **Done before charter** in `charter synth`.

Running units in a chain

Say *"/charter-chain <unit> <your words>"*. This session starts it on `chain/<id>` and watches; fresh sessions work, each checking the last, handing over at 25% of context (cap 30%). Codex reviews each rail item; the next session answers each finding. Ends `complete`, `halted` (its reason is your question) or `stopped`. Keep out of the clone meanwhile. This machine's `/supervise` road: back.

O **It waits on you:** `! python3 .charter/bin/charter owner run` runs the acts in your shell (`--only N` reruns one), then `/resume-chain ok` closes the lines whose acts passed; `skip <note>` closes none; a halt: `/resume-chain <note>`. Only you can resume it.

`charter chain status` names each group's cleared commit: merge up to one, never by squash. `chain report` is the PR comment.

Sources: CharterAdmin `workflows/Migrate.md`; `docs/migration.md` Paths C-E; `docs/daily-use.md` § Running units in a chain; `docs/design/continuous-chain.md`.

Once a week

`docs/daily-use.md` § The owner's weekly look

- `charter next`: **Waiting on the owner** lists proposals, finished rails, files staged for you. `charter owner`: that queue alone, with each command.
- `charter synth`, no `--write`: adds gate activity.
- `charter log`: 30 days of refusals and warnings. A warning that keeps catching real problems is evidence for a gate; one that fires on nothing is noise.
- No new gates before two or three projects finish a full cycle (your ruling, 2026-09-24).
- `charter config`, then opt in only when the project can bear it: `probes.at_commit`/`in_ci` when probes run offline and fast; `probes.base_must_fail` when they reliably fail first; `review.required` with an independent reviewer; `rails.required` when every open unit names a rail.

Charter reference

A unit, evidence, refusals, every command, the road.

O owner · A agent · refused · warns · writes nothing

A unit, `.taskets/<id>.md`

```
---
id: 20260925-greet
title: A greeting function
status: next (1)
scope: S1 (2)
rail: r1-greet
perimeter: [app/**] (3)
kind: work
---
## Acceptance (4)
- [ ] AC-1: greet() returns hello
  — probe: `python3 -c "…"`
- [ ] AC-2: (guard) old API intact
  — probe: `pytest tests/api -q`
## Evidence (5)
- a76f695 adds greet (app/greet.py:1-2)
- `$ python3 -c "…"` → exit 0
## Log (6)
- 2026-09-25 09:00 started; baseline
  AC-1 → exit 1
## Outside perimeter
```

1. `now next later blocked done`; closed without evidence, honestly: `parked`, `skipped`. Comments go on their own line, never after a value.
2. `scope:` an in-scope item or an approved change (client track). `rail:` an open item of `charter/rails.md`.
3. Paths it may touch. Outside them, a note under `## Outside perimeter`: refused at commit on client, warns on prototype.
4. One binary probe per line, written first, failing before the work; once started, a changed line needs a dated log line. `— owner: check|do` before the probe: only you close it.
5. A commit, a `path:line`, the `$ command` lines `charter probe` printed. With `review.required`: `- review: <who, model> — <verdict> — <anchor>`.
6. Append-only and dated. The next session resumes from here, not from the transcript.

Sources: `package/charter/templates/unit.md`; `package/.claude/rules/charter.md:7-25`; `TERMINAL_STATUSES` in `package/.charter/lib.py`.

Evidence the done gate accepts

- counts: a commit SHA that resolves
- counts: `$ command` → exit 0, as run
- counts: `path:line` of the change, beside a commit or run
- counts: every `- [x]` ticked after its probe passed
- refused: a path or a URL alone
- refused: "it works", "tests pass"
- refused: an anchor that does not resolve
- refused: one box left `[ ]`

A probe that passes before the work warns (refused with `probes.base_must_fail`). The gate sees that evidence resolves, not that it proves the claim: that is the reviewer's job, and yours.

Two tracks

	client	prototype
Scope	`charter/scope.md` required, gate hard	none, gate off
Perimeter note	hard at commit	warns
Anchored reports	hard in CI	warns
Done, size, decisions	hard on both; rails optional on both	

A prototype that gains a client: O writes `charter/scope.md`, sets `"track": "client"`.

Sources: `docs/gates.md` § Done needs evidence; `docs/method.md` § Two tracks.

When a gate refuses

You see	Who	Do this
`charter/done block` … `unchecked`	A	Run each probe, tick the box, cite the run under `## Evidence`; or keep the status open.
`charter/done block` … `names no commit … and no $ command`	A	Add the SHA and the `$ command` lines `charter probe <id>` prints. No flag overrides done: without evidence, `parked` or `skipped`.
`charter/protected block [<path>]` at commit, or a denied Edit of an owner-only file	A → O	Leave it staged; `charter handover` prints the owner's command. In session, propose instead: a change request, a successor decision, a draft outside the project.
Out-of-scope path (client track)	A → O	Write `charter/changes/<slug>.md` from `charter/templates/change.md`, `status: proposed`; the owner approves.
Frozen file grew past its limit	A → O	Move code out; shrinking always passes. Or the owner opens a `kind: split` unit or a `size_exceptions` entry.
Retired decision cited, or a decision's `check:` fails	A → O	Cite the successor the message names; or write a new decision with `supersedes:`, and the owner retires the old one.
`charter/rails warn` … `names no rail` (refused with `rails.required`)	A → O	Set `rail: <open id>`. Work the road does not name waits: the owner adds a rail first.
A chain session past the 30% cap: all refused but the handoff, unit files, git, the CLI	A	Hand over: the handoff as in `charter/templates/handoff.md`, then `charter chain next`.
`/resume-chain <note>`: owner lines still open	O	`/resume-chain ok` after `charter owner run`, or `skip <note>`; `AC-n` names one line.
Stop hook: a done claim with no commit, `path:line` or run	A	Cite one, or say plainly what is not done. It blocks once per turn; an anchor it cannot resolve only warns.

Sources: `docs/install.md` § When a gate refuses; `docs/gates.md` § The gates, § Chain budget and hand-over, § The owner's resume; each checked against 0.4.11.

In a project

Command	What it does	Writes
`charter next [--limit N]`	Where it stands, what comes next	nothing
`charter config` · `version` · `unit show`	Track, opt-ins; version; unit	nothing
`charter log [--days N] [--verbose]`	What the gates did, 30 days	nothing
`charter owner [--json]` · `handover`	Your queue; your commit, again	nothing
`charter chain start|status|stop|report`	Via `/charter-chain`	a branch
`charter synth [--write]`	The synthesis; refused if unanchored	`progress.md`
`charter unit start <id>` · `unit stop`	Attribute edits and cost	git dir
`charter probe <id> [--timeout S]`	Runs probes, prints `$` lines	gate log
`charter check --staged|--all [--base REF]`	Every gate; `--all` runs `check:`	gate log
`charter anchors <file> [--pr]`	Every claim has an anchor	gate log
`charter ledger fold` · `add --human-minutes N`	Sessions' cost; your time	`ledger.jsonl`
`charter decisions index` · `setup`	Decision index; git hook	`INDEX.md`, git
O `charter rule|approve|rail … [--dry-run]`	Rule, approve, set a rail	one commit
O `charter owner run [--only N]`	A wait's acts, your shell	git dir

Sources: `docs/package.md` § Commands; `cmd_*` in `package/.charter/bin/charter`; `owner.py`, `owner_acts.py`, `chain.py` in `package/.charter/`.

From the charter checkout

```
charter-pkg survey $P --track T --out DIR [--test-cmd C] [--keep-clone]
charter-pkg install $P --track T [--dry-run]
charter-pkg upgrade $P [--dry-run]
charter-pkg uninstall $P [--dry-run] [--purge-state]
charter-pkg status $P
charter-pkg fleet [--root DIR] [--upgrade]
```

O **Only the owner changes** `charter/scope.md`, `rails.md`, `config.json`, `.charter/**`, `.claude/settings*.json`, `.github/workflows/charter.yml`; a new id in `legacy-taskets.txt`; a change's approval fields; a decision's `status`, `superseded_by`, and an active one's `check:`, `binds:`; opening a `kind: split` unit; closing an `— owner:` line. And every commitment: what is sold, price, dates, deploys, production data, releases, merging.

A **The agent never** sets `CHARTER_OWNER`, passes `--no-verify` (or `--force`, `--replace-hooks` without your word), writes a refused file through Bash, runs `git add -A` or `charter owner run`, resumes a chain, pushes, merges or deploys.

Skills in the project `charter-unit` *"start unit" "resume" "finish the unit"*; `charter-decision` *"record the decision" "this is out of scope" "change request" "lesson learned"*; `charter-progress` *"where does this stand" "status for the client"*; `charter-chain` *"/charter-chain" "chain status" "stop the chain"*.

Personal layer `CharterAdmin`: *"charter status" "upgrade charter on X" "migrate X"*; decisions → Slate. `package X available`: X is the branch `~/Projects/charter` has out.

Sources: `package/.claude/rules/charter.md:28-47`; `docs/install.md` § Rules for the agent; `package/.claude/skills/`; `personal/claude-home/`.

Personal layer: work the road unattended

This machine only, needs tele-claude; it merges each PR it verifies. The package's own way is the chain (front).

`/supervise --road [--context-cap 25] [--max-units N]`: the units in `charter next` order, one run each; verifies each PR, merges it via `/reviewPR`, closes the unit. Stops on a problem, the cap, an empty road.

`/supervise --road --infinite [<project>]`: stops only for you. Hands over at the cap, waits out limits, parks a blocked unit with what builds on it. Pushes its own `.taskets/`-only commits.

With `--infinite`, all optional; left out, the first two are asked once at the start

`--week-target 70`	The week's usage it may reach, paced: a day's share at once, then even. **70** a working week, **90** a week off; none: the 95% cap only.
`--stop-at-rail <id>`	End before this rail; offered: `after-active`, `before-parked`, the whole road.
`--max-units 2`	A first trial on a new project.
`--gate "<cmd>"`	One command judges every merge; else worked out per unit (decisions' `check:`, its tests, what imports the change). Files red on main at the start cannot fail it.

Start sweep: `grant` a kind of change, `review` (PR parked for you), `keep a gate: human`, `last`, `hold`. Unit keys: `on_block: park|stop`, `road_grant`, `road_review`.

Running: `Road.py tally report --file <state>`; say *"stop the road"*; `--resume <state>`. A cron keeper wakes it.

Always: no launch past 85% of the 5-hour window, 95% of the week, or 00:00–06:00 Paris.

Sources, `~/AI_tooling/PAI/generic/claude/` at 1605b02: `Commands/supervise.md:37-62`; `Skills/SupervisedMaster/` `INFINITE_CHEATSHEET.md`, `workflows/InfiniteRoad.md`.

Where it stops: evidence is checked for existence, not truth; the scope gate matches paths; a Bash edit skips the session hooks; gates fail open on their own errors: one error line and a pass is a charter bug, see `charter log` (`docs/gates.md` § What a gate cannot see).

Checked against 0.4.11 and a rehearsal, 2026-10-06. `docs/cheatsheet.html`